CAPSTONE EVIDENCE & TECHNICAL AUDIT

SAFESPACE CAPSTONE EVIDENCE AND TECHNICAL AUDIT REPORT

SAFESPACE: A Mobile and Web-Based Abuse Reporting and Response System for Aurora Province

Aurora State College of Technology

School of Information Technology

Report generated: 2026-10-13

Table of Contents

What was actually accessible 4
Existing documentation found, and how it's being treated 5
Evidence-sourcing convention used throughout this audit 5
Project directory 6
Mobile application 6
Web dashboard (admin) 7
Backend (Supabase) 8
Database (summary — full inventory in Deliverable 3) 9
Authentication (summary — full workflow verification in Deliverable 2) 10
Authorization 10
Testing 10
Deployment 10
Documentation 11
1. Registration, Email Verification, Login, Password Recovery 12
2. Role-Based Access Control and Row-Level Security 13
3. Abuse Incident Reporting (Survivor Mobile) 14
4. Emergency SOS 16
5. Case Assignment, Status Transitions, and Escalation 17
6. Shelter Requests and Shelter Administration 19
7. Police Officer Referrals and Access Restrictions 20
8. AI Support, Safety Triage, and Human-Support Handoff 22
9. Notifications and Delivery Mechanisms 24
10. Reports, Analytics, Audit Logs, Backups, and Recovery 25
Unresolved questions / structural and documentation gaps 27
Purpose and scope 28
Reconciliation table 29
What this means for the manuscript 30
What was explicitly not re-litigated in this document 31
This is a draft working document, not an approved evaluation instrument 32
How to read the matrix below 32
Functional Suitability 33
Performance Efficiency 33
Compatibility 34
Usability 34
Reliability 34
Security 35
Maintainability 35
Portability 35

What this document does not do 36
Critical 36
High 37
Medium 38
Low / informational 39
How to use this register going forward 39

SECTION 00

SafeSpace Capstone Audit — Scope, Access, and Evidence Sources

Status: in progress — first pass
Source document date: 2026-10-12
Source file: `docs/capstone-evidence/00-audit-scope-and-access.md`

This is a **read-only** inspection. No source code, database schema, RLS policy, Supabase configuration, environment variable, deployment setting, or production data was modified to produce this document. All findings below come from direct inspection of the repository as it exists on disk at `C:\Users\Jastine\Desktop\safeher-mobile`, run on 2026-10-12.

What was actually accessible

Location	Accessible?	Notes
`mobile/` (full source tree)	Yes	Expo/React Native app. `node_modules` excluded from inspection (not source).
`admin/` (full source tree)	Yes	Vite/React web app.
`shared/`	Yes	One file: `database.types.ts`.
`supabase/` (migrations, functions, email templates)	Yes	32 numbered SQL modules + `00_schema.sql` + 6 Edge Functions.
`.env` files (both apps)	Yes, **names only**	Confirmed which variables are *referenced*; values were never read or reproduced, per this audit's own confidentiality rule and the project's own `.env.example` convention.
Live Supabase project (database, Auth, Storage)	**No**	No database credentials, dashboard access, or live query capability in this environment. Every database claim below is sourced from migration *files*, not a live schema introspection.
Google Play Console / EAS dashboard	No	Not applicable to this audit (no build or deployment action taken).
`SafeHer-System-Inventory.pdf` (repo root)	**No**	PDF rendering is unavailable in this environment (`pdftoppm`/poppler-utils not installed). File exists (764 KB, dated 2026-09-09) but its contents are **not** reflected anywhere in this audit. A team member with normal PDF access should review it separately and reconcile it against these findings.

root)	Yes	Read in full. Treated as a **prior, team-authored** evidence source — see below.
Physical devices, Play Store listing, any human evaluator	No	Not applicable to a code audit.

Existing documentation found, and how it's being treated

Three pre-existing documents were found at the repository root. None of them are treated as current fact without cross-checking against the actual source/schema — this is the central discipline this task requires.

1. `README.md` (repo root) — **Severely stale.** Describes "exactly 14 tables," "4 role enum values," "2 Edge Functions," and `ANTHROPIC_API_KEY` as the AI provider. Direct inspection (below) found 32 tables, 10 historical role-enum values (8 currently active), and 6 Edge Functions. This file appears to date from the project's earliest commits and has not been updated since. **Do not cite this file as a current source for the manuscript** — it is preserved here only as a historical artifact.
2. `SafeHer_System_Documentation_Report.html` (repo root, dated 2026-09-09) — A substantive, well-constructed capstone documentation report, authored under the system's prior name ("SafeHer") and a narrower VAWC-only framing. Its **author list matches the actual git commit author** for this repository (Ramirez, Danica Jane B.), so this is treated as a **genuine prior artifact of this research team**, not a third party source. It reports (as of its writing): 9 administrative roles, 26 survivor-facing mobile screens, 18 schema modules, 5 Edge Functions, 8 municipalities covered. Direct inspection today confirms the *architecture and methodology* descriptions in this report remain substantively accurate, but the **counts are now stale** — the project has grown to 32 schema modules and 6 Edge Functions since this report was written (see Deliverable 1 below). It also documents a real, specific RLS verification run (§6, "SOS & case referral pipeline") performed against disposable test accounts — this is **historical evidence of testing having occurred**, not something this audit re-ran or re-verified itself. This report also predates the SafeSpace rebrand and the gender-inclusive scope statement in the current master prompt — any manuscript text drawn from it must be updated for both.
3. `SafeHer-System-Inventory.pdf` — inaccessible, see table above.

Evidence-sourcing convention used throughout this audit

Every claim in the deliverables that follow is tagged with where it came from, using these categories (per the governing master prompt's own evidence register requirement):

- **[CODE]** — directly read from application source in this pass.
- **[SCHEMA]** — directly read from a `supabase/*.sql` migration file.
- **[DOC-2026-09]** — sourced from the existing HTML report above, not independently re-verified in this pass.
- **[SESSION]** — verified earlier in this same working session (same day, same repository state), via direct code reads, `tsc`/build runs, or `expo-doctor` output — not re-run in this specific pass, but not stale either (no intervening changes to the relevant files).

- **[UNVERIFIED]** — a claim from prior documentation or planning that this pass could not confirm against current code/schema.
- **[GAP]** — something this audit explicitly could not determine and flags as needing the team's attention.

Nothing below is marked as tested, passing, or production-verified unless a concrete command, file, or prior documented test run is cited.

SECTION 01

Deliverable 1 — Project Structure and Technical Inventory

Status: complete (first pass)
Source document date: 2026-10-12
Source file: `docs/capstone-evidence/01-deliverable-1-project-structure.md`

Status: **COMPLETE** for the purpose of this first pass. Re-check before manuscript submission if further development occurs after this date.

Project directory

Root: `C:\Users\Jastine\Desktop\safeher-mobile` — no root-level `package.json` **[CODE]**. The repository is four independent applications/packages, not a monorepo with shared tooling:

```
safeher-mobile/
├── mobile/     React Native (Expo) survivor-facing app
├── admin/      React + Vite administrative web dashboard
├── shared/     one file: database.types.ts (hand-aligned copy, not a build-linked package)
└── supabase/   SQL migrations, Edge Functions, email templates
```

GitHub remote: `https://github.com/safeheradvocacy/SafeHer-Mobile.git`, branch `main` **[SESSION]**.

Mobile application

- **Entry point**: `mobile/index.ts` → `mobile/App.tsx` **[CODE]**.
- **Navigation root**: `mobile/src/navigation/RootNavigator.tsx` — branches between `AuthNavigator` (unauthenticated), `AppNavigator` (authenticated), and a dedicated `RecoveryNavigator` (password-reset deep link, kept separate specifically so a recovery session can never silently drop a user into the full app with their old password still active) **[CODE]**.
- **Screens**: `mobile/src/screens/` — 27 `.tsx` files across 12 subdirectories (`announcements`, `assistant`, `auth`, `caseAccess`, `cases`, `home`, `incident`, `legal`, `notifications`, `profile`, `shelters`, `sos`, `verification`). Some files contain more than one logical screen (e.g. `auth/PasswordRecovery.tsx` exports both `ForgotPasswordScreen` and `ResetPasswordScreen`; `auth/SplashOnboardingWelcome.tsx` exports the splash/onboarding/welcome sequence) **[CODE]**. Prior documentation **[DOC-2026-09]**

counted 26 distinct navigable screens; this is consistent with the file count above and was not re-derived screen-by-screen in this pass — flagged as **[GAP]** for an exact re-count if the manuscript needs a precise figure.

- **Auth/session**: `mobile/src/contexts/AuthContext.tsx` (session state, sign-in/up/out, OTP verification, password recovery/change) and `mobile/src/lib/supabase.ts` (Supabase client init) **[CODE]**.
- **Session storage**: `mobile/src/lib/secureSessionStorage.ts` — a chunked `expo-secure-store` adapter (Android Keystore / iOS Keychain), with a one-time migration path from the prior `AsyncStorage`-backed storage **[SESSION]**.
- **Deep links**: `mobile/src/lib/deepLinks.ts` (auth callback links, Client Case Access QR links) **[CODE]**.
- **Emergency SOS logic**: `mobile/src/lib/useSosTrigger.ts` (state machine: idle → locating → located/location-unavailable → saving → done/error), `mobile/src/lib/sosLocation.ts` (location preflight/acquisition), UI in `mobile/src/screens/sos/SosScreen.tsx` and `mobile/src/screens/home/SosConfirmSheet.tsx` **[CODE]**.
- **AI assistant**: `mobile/src/lib/useSupportChat.ts`, `mobile/src/screens/assistant/SafeHerAssistantScreen.tsx` — calls `supabase.functions.invoke('safeher-chatbot', ...)`; no AI provider key present anywhere in `mobile/src` (repo-wide grep for `sk-`, `openai`, `anthropic`, `groq`, `apiKey` near AI code returned no privileged key) **[SESSION]**.
- **Push notifications**: `mobile/src/lib/pushNotifications.ts`, `mobile/src/lib/usePushNotifications.ts` — explicitly self-documented in code as inert without an EAS `projectId` and inert inside Expo Go; separate from the database-backed `notifications` table **[SESSION]**.
- **App configuration**: `mobile/app.config.ts` (dynamic Expo config — converted from a static `app.json` during this working session's Android release-prep pass), `mobile/eas.json` (build profiles) **[SESSION]**.
- **Dependencies**: `mobile/package.json` / `mobile/package-lock.json` — Expo SDK `~57.0.27`, React `19.2.3`, React Native `0.86.3`, TypeScript `~6.0.3`, `@supabase/supabase-js@2.45.4` (pinned exact, not a `^` range — see "The one real bug this build caught" in the root `README.md`, which remains a plausible historical note even though the rest of that file is stale) **[SESSION]**.
- **Automated tests**: **none found.** No `*.test.*` / `*.spec.*` files and no Jest/testing-library config anywhere under `mobile/src` **[SESSION, confirmed via direct filesystem search]**.
- **Anomaly**: `mobile/safeher-mobile/` is a separate, unrelated, committed Vite+React scaffold (its own `package.json`, React 19, `oxlint`) with no connection to the real Expo app — appears to be a very early false-start committed in "SECIND COMMIT" and never developed further or removed. Flagged for the team to decide whether to delete; not touched by this audit. `mobile/supabase/.temp/` is Supabase CLI metadata (project ref only, not a secret) accidentally committed from running the CLI in the wrong directory — low-priority housekeeping item **[SESSION]**.

Web dashboard (admin)

- **Entry point**: `admin/src/main.tsx` → `admin/src/App.tsx` (React Router `BrowserRouter`, route guards via `ProtectedRoute`) **[CODE]**.

- **Pages**: `admin/src/pages/` — 52 `.tsx` files across 20 subdirectories, including role-specific dashboards (`dashboard/ProvincialDashboard.tsx` , `MunicipalDashboard.tsx` , `TechnicalAdminDashboard.tsx` , `LawEnforcementDashboard.tsx` , `ShelterAdminDashboard.tsx` , `SupportExpertDashboard.tsx` , routed via `dashboard/DashboardRouter.tsx`), case management (`cases/CaseListPage.tsx` , `CaseDetailPage.tsx`), SOS monitoring (`sos/SosMonitoringPage.tsx`), shelters (5 pages under `shelters/`), law enforcement (`lawEnforcement/SosReferralsPage.tsx` , `CaseReferralsPage.tsx`), police personnel administration (`police/PoliceManagementPage.tsx`), verification queues (`verifications/`), reports and analytics (10 files under `reports/` , one `*Analytics*` / `*Reports*` pair per role), announcements, barangay inter-agency requests, backups, audit log, roles/permissions reference page, and the public landing page (`public/`) **[CODE]**.
- **Auth/session**: `admin/src/contexts/AuthContext.tsx` , `admin/src/lib/supabase.ts` , route protection in `admin/src/routes/ProtectedRoute.tsx` **[CODE/SESSION]**.
- **Permission model**: `admin/src/lib/permissions.ts` — the single source of truth the sidebar (`admin/src/layout/AppLayout.tsx`) and route guards both derive from, explicitly documented in its own header as *"a UI-layer convenience, not the actual security boundary"* — RLS is **[SESSION, read in full multiple times this session]**.
- **PDF/report generation**: `admin/src/lib/reportDocument.ts` , `governmentReportDocument.ts` , `analyticsReportDocument.ts` — real vector PDF generation via `jspdf` / `jspdf-autotable` , not screenshots **[SESSION]**.
- **Dependencies**: `admin/package.json` — React 18.3.1, Vite 5.4.5, TypeScript 5.5.4, `@supabase/supabase-js@2.45.4` (same pinned version as mobile), `react-router-dom@6.26.2` , `jspdf` , `chart.js` , `@react-google-maps/api` **[SESSION]**.
- **Automated tests**: **none found** — same confirmation method as mobile **[SESSION]**.
- **Deployment**: `admin/vercel.json` (SPA rewrite config) — prepared for Vercel during this working session; **not confirmed deployed to a live URL** (no Vercel account access in this environment) **[SESSION]**.

Backend (Supabase)

- **Base schema**: `supabase/00_schema.sql` — original tables, enums (`user_role` originally 4 values, `case_type` originally 6 values, etc.), base RLS **[SCHEMA]**.
- **Migrations**: 32 numbered module files, `module1_auth.sql` through `module32_police_management.sql` , plus `module1b_grants_and_survivors_rls.sql` and `module3b_service_role_grants.sql` (both exist specifically because a live test surfaced a missing grant after the fact — documented in `[DOC-2026-09]`) and one markdown file, `module3_admin_provisioning.md` **[SCHEMA, counted directly]**. This is **14 more modules than the 18 the 2026-09 report described** — the backend has grown substantially since that report was written.
- **Edge Functions**: `supabase/functions/` — 6 functions: `create-admin-user` , `admin-user-metadata` , `safeher-chatbot` , `send-push-notification` , `backup-create` , `backup-restore` , plus a `_shared` helper directory **[SCHEMA]**. This is one more than the 5 the 2026-09 report described (`admin-user-metadata` was added since). Every function pairs a caller-JWT-scoped client (to verify identity) with a service-role-key-holding client (for the privileged action itself) — confirmed by direct reads of `create-admin-user/index.ts` and `admin-user-metadata/index.ts` earlier this session **[SESSION]**.

- **Email templates**: `supabase/email-templates/` — a custom-built template system (shared brand/layout partials + 4 templates: confirm-signup, reset-password, security-code/OTP, change-email) with a `build.mjs` generator. Explicitly documented in its own `README.md` as requiring **manual deployment** into the Supabase Dashboard's auth-email template slots — **not auto-deployed by any script [SESSION]**.

Database (summary — full inventory in Deliverable 3)

- **32 tables**, enumerated directly from every `create table` statement across `00_schema.sql` and all 32 modules: `announcements`, `audit_logs`, `backup_schedule`, `barangay_action_requests`, `barangays`, `case_access_tokens`, `case_assignments`, `case_client_links`, `case_referrals`, `case_reports`, `case_services`, `case_status_history`, `chat_messages`, `chat_sessions`, `evidence`, `expert_support_requests`, `faqs`, `law_enforcement_referrals`, `municipalities`, `notifications`, `push_tokens`, `saved_reports`, `shelter_referrals`, `shelter_requests`, `shelters`, `sos_alerts`, `survivors`, `system_backups`, `system_restores`, `terms_acceptances`, `user_profiles`, `user_verifications` **[SCHEMA, grep-verified, reproducible]**.
- `user_role` **enum** — started with 4 values in `00_schema.sql` (`survivor`, `municipal_admin`, `provincial_admin`, `super_admin`), grew via `alter type user_role add value` in modules 3, 4, 6, 7, 9, and 11, adding `barangay_admin`, `shelter_admin`, `support_expert`, `shelter_staff`, `law_enforcement`, `provincial_staff` respectively — **10 enum values total, ever added [SCHEMA]**. `shelter_staff` was later retired (`module24_retire_shelter_staff_role.sql` — role consolidation into `shelter_admin`) and `super_admin` is no longer a creatable target for new accounts (enforced in `create-admin-user/index.ts`, treated identically to `provincial_admin` everywhere it still appears) **[SESSION]**. **Currently active, assignable roles: 8** — `survivor`, `barangay_admin`, `municipal_admin`, `provincial_staff`, `provincial_admin`, `shelter_admin`, `support_expert`, `law_enforcement`.
- **Storage buckets** (from `insert into storage.buckets` statements): `identity-documents` (private, 8 MB limit, module10), `evidence` (private, 8 MB limit, module29), `system-backups` (private, 50 MB limit, module16) **[SCHEMA]**. **Resolved (was a [GAP] as of 2026-10-12, first pass):** the `avatars` bucket is **not** created by any SQL migration, and this is explicitly confirmed by the project's own code — `module10_identity_verification.sql` line 338 states outright: *"avatars/evidence were both created directly in the Supabase dashboard and have no SQL anywhere."* That comment is itself now partially stale: `evidence` **was** later brought under migration tracking (`module29_evidence_storage_limits.sql`), confirmed by a full-repository grep for `storage.buckets` entries. `avatars` was **not** — no later module (11 through 32) adds it either, confirmed by the same grep. It is used in both apps as a **public** bucket via `getPublicUrl()` (`mobile/src/screens/profile/ProfileScreen.tsx`), which is an appropriate access level for non-sensitive profile photos — this is not a security concern. **It is, however, a real deployment-reproducibility gap**: a fresh deployment of this project from the repository alone, onto a new Supabase project, would be missing this bucket (and whatever access policies it has) entirely, since nothing in version control creates it. The bucket's *exact* RLS/access policy text still cannot be confirmed without live Dashboard access **[CODE + SCHEMA, confirmed 2026-10-13]**.

Authentication (summary — full workflow verification in Deliverable 2)

- Supabase Auth, email/password, **OTP-code email verification** (not link-based — 8-digit code, `supabase.auth.verifyOtp({..., type: 'email'})`), PKCE flow for the mobile app's deep-link exchanges, separate password-recovery deep-link handling **[SESSION, read directly in `AuthContext.tsx` this session]**.
- `super_admin` as a role label is deprecated for new accounts but remains a live enum value — see above.

Authorization

- Role-based access control at the application layer (`admin/src/lib/permissions.ts`) **layered on top of** PostgreSQL Row-Level Security, which both the project's own code comments and `[DOC-2026-09]` identify as the actual enforcement boundary — the permissions file is explicit that it is "a UI-layer convenience, not the actual security boundary" **[SESSION]**. Full role/permission matrix is Deliverable 3D (not yet produced in this pass).

Testing

- **No automated test suite exists in this repository** — confirmed by direct filesystem search in both `mobile/` and `admin/` **[SESSION]**.
- The only documented testing evidence found is the **manual, scenario-based RLS verification run** described in `SafeHer_System_Documentation_Report.html` §6 (SOS/case-referral pipeline, 8 scenarios, disposable test accounts, cleaned up afterward) — this is **prior evidence from the team's own earlier report**, not something this audit re-ran **[DOC-2026-09]**.
- That same report's own Recommendations section (§7) already recommends converting this into an automated regression suite — **this has not happened** as of this inspection **[SESSION + DOC-2026-09, consistent]**.
- Within this working session, `npx tsc --noEmit` and `npx expo-doctor` were run repeatedly against the mobile app (clean/21-of-21 passing at last run) and `npx tsc -b && vite build` against the admin app (clean) — these are **static/build-correctness checks, not functional or security tests [SESSION]**.

Deployment

- **Mobile**: Expo-managed workflow (no committed `/ios` or `/android` native folders — confirmed via `.gitignore`). EAS Build is configured (`mobile/eas.json` — `development` / `preview` / `production` profiles) but **no build has actually been produced** — this environment has no EAS account authentication **[SESSION]**.
- **Admin**: `admin/vercel.json` prepared for a Vercel deployment this session; **no evidence of an actual live production URL** — this environment has no Vercel account access **[SESSION]**.
- **Backend**: a Supabase project exists and is linked (confirmed via `supabase/.temp/project-ref` and `mobile/supabase/.temp/linked-project.json` — project name "SafeHer", a single project, no separate

staging instance found or referenced anywhere) **[SCHEMA]**. Whether all 32 migration modules have actually been **applied** to that live project cannot be confirmed without live database access — **[GAP]**.

Documentation

- `README.md` (root) — stale, see above.
- `SafeHer_System_Documentation_Report.html` (root) — substantive, partially stale on counts, see above.
- `SafeHer-System-Inventory.pdf` (root) — inaccessible in this environment, see above.
- `supabase/module3_admin_provisioning.md` — a markdown note alongside the SQL modules, not yet read in this pass — **[GAP, next step]**.
- `supabase/email-templates/README.md` — read earlier this session, documents the manual Supabase Dashboard deployment requirement for email templates **[SESSION]**.
- No requirements specification, ERD diagram file, or user manual was found in the repository itself — **[GAP]**. (The 2026-09 HTML report contains an inline architecture diagram and references an "ERD," but no separate ERD file exists in the repo.)

SECTION 02

Deliverable 2 — Feature Verification Matrix

Status: All 10 priority feature areas now covered; runtime/live verification still outstanding for every one
Source document date: 2026-10-12 (sections 1–4); continuation pass 2026-10-13 (sections 5–10)
Source file: `docs/capstone-evidence/02-deliverable-2-feature-matrix.md`

Status: **All priority feature areas named by the governing master prompt are now covered** — Registration & Authentication, RBAC/RLS, Abuse Incident Reporting, Emergency SOS (first pass, 2026-10-12), and Case Assignment/Escalation, Shelter Requests/Administration, Police Officer Referrals, AI Support/Safety Triage, Notifications, and Reports/Analytics/Audit/Backups (this continuation pass, 2026-10-13). "Covered" means: the actual source code and SQL were read directly and the required fields below were filled in from that evidence. It does **not** mean every workflow has been confirmed to work correctly at runtime against the live project — per the governing instruction, *"Do not infer successful runtime behavior from source code alone. Where no test evidence exists, state that the behavior remains unverified,"* every section below says explicitly, per-row, where that is the case. The consolidated list of what still needs live/runtime verification is collected in `05-unresolved-evidence-and-testing-register.md`, not repeated field-by-field in the "Unresolved questions" note below, which is now reserved for structural/documentation gaps rather than per-feature test needs.

Implementation-status vocabulary used below, exactly per the governing master prompt's six categories: **Implemented in code** / **Integrated with the backend** / **Tested successfully** / **Partially implemented** / **Planned but not implemented** / **Not verified (insufficient evidence)**.

---

1. Registration, Email Verification, Login, Password Recovery

Field	Detail
Intended user/role	Survivor (the only role with a mobile account registration flow — confirmed no other role self-registers; admin accounts are provisioned by an existing admin, see `create-admin-user` Edge Function)
Purpose	Let a survivor create an account, confirm control of their email, sign in, recover a forgotten password, and change their password while signed in
Mobile screens	`mobile/src/screens/auth/RegisterScreen.tsx`, `EmailVerificationScreen.tsx`, `LoginScreen.tsx`, `PasswordRecovery.tsx` (ForgotPassword + ResetPassword), `mobile/src/screens/profile/ChangePasswordScreen.tsx`
Source files	`mobile/src/contexts/AuthContext.tsx` (all Supabase Auth calls), `mobile/src/lib/deepLinks.ts` (recovery/verification deep-link parsing), `mobile/src/lib/supabase.ts` (client config: `flowType: 'pkce'`, `detectSessionInUrl: false`)
Backend	`supabase.auth.signUp` / `signInWithPassword` / `verifyOtp` / `resetPasswordForEmail` / `updateUser` (Supabase Auth, not a custom table); `public.user_profiles` and `public.survivors` rows are created server-side by an `on_auth_user_created` trigger (`supabase/module1_auth.sql`), reading signup metadata — **not** written directly by the client
Workflow	Register (collects name/phone/address + consent) → `signUp()` → Supabase sends an 8-digit OTP code (not a link) → `EmailVerificationScreen` calls `verifyOtp(email, code, type:'email')` → on success, a real session exists → terms-acceptance row and deferred survivor-info fields are best-effort backfilled → session picked up automatically by `AuthContext`'s `onAuthStateChange` listener, routing into the authenticated app
Required permissions	None (pre-authentication)
Validation / error handling	`AuthContext.tsx` wraps every one of these calls in try/catch and maps raw Supabase errors to specific, non-technical messages (e.g. distinguishes "email not confirmed," "rate limited," "invalid credentials," "account already exists" without revealing which accounts exist) — confirmed by direct read of `friendlyAuthError()` **[SESSION]**. Password-recovery session handling is specifically hardened: `RootNavigator.tsx` checks a `pendingPasswordRecovery` flag **before** its normal authenticated-app routing, so exchanging a recovery link's code can never silently drop a user into the full app with their old password still active **[SESSION]**.
Existing test evidence	**None found as an automated test.** The 2026-09 documentation report does not describe a specific test run for this flow (its documented test run covers the SOS/referral pipeline, not registration).

status	**Implemented in code, integrated with the backend.** Not independently confirmed as "tested successfully" by this audit — see below.
Known limitations	OTP length (8 digits) must match the Supabase project's own Email OTP Length dashboard setting — this is an external configuration dependency that cannot be verified from source code alone **[GAP]**. "Remembered email" (not password) is cached in plain `AsyncStorage` (`LoginScreen.tsx`) — low sensitivity, noted for completeness.
Additional testing required	A real registration → OTP email receipt → verification → login round trip, performed manually against the live project with a disposable test account, with the actual screen states captured as evidence (screenshot or screen recording). This audit did not perform this — no live project access. Password-reset email delivery and link/code handling likewise needs a real round trip.

---

2. Role-Based Access Control and Row-Level Security

Field	Detail
Intended user/role	All 8 active roles (see Deliverable 1)
Purpose	Ensure each role can only see/act on the data its jurisdiction and function authorize — enforced at the database layer, not merely hidden in the UI
Web pages	Every admin page is gated two ways: (1) `admin/src/routes/ProtectedRoute.tsx` with a hardcoded `requireRole` array per route in `admin/src/App.tsx` ; (2) the sidebar nav in `admin/src/layout/AppLayout.tsx` filters by permission key from `admin/src/lib/permissions.ts`
Source files	`admin/src/lib/permissions.ts` (UI-layer role→permission map, **explicitly documented in its own header as not the real security boundary**); `supabase/00_schema.sql` + all 32 modules (actual `create policy` statements)
Backend	PostgreSQL RLS policies on every sensitive table, plus `SECURITY DEFINER` helper functions (e.g. `is_provincial_admin()` , `admin_can_access_case()` , `admin_can_access_profile()`) used specifically to avoid policy-recursion errors — confirmed present across `module2_case_data_rls.sql` , `module3_barangay_hierarchy.sql` , `module11_provincial_staff_rbac.sql` , and others read directly this session
Workflow	A request (from either client) goes through PostgREST, which evaluates RLS policies using the caller's JWT — a row is only returned/writable if a policy's `using` / `with check` clause evaluates true for that `auth.uid()` . The frontend `permissions.ts` map decides what to *show*, but a direct, crafted API call bypassing the UI is still subject to the same RLS the UI respects — this is the architecture's stated design principle, confirmed by reading the policy definitions, not merely the claim
Required permissions	N/A — this *is* the permission system

ng	A denied RLS read/write returns an empty result set or a `42501` /permission-denied error, not a 200 with unauthorized data — confirmed by reading policy logic directly; **not independently re-tested live by this audit pass**
Existing test evidence	The 2026-09 report's §6 live-verification table (8 scenarios: unassigned-record denial, cross-role denial, unauthenticated denial, guessed-ID denial — all reported PASS) is the only concrete test evidence found for this area, and it covers only the SOS/case-referral slice, not the full role matrix **[DOC-2026-09]**.
Implementation status	**Implemented in code, integrated with the backend.** The specific scenarios in the 2026-09 report are the only ones with documented test evidence; the broader role/permission matrix (9→8 roles × ~15 tables) has **not** been exhaustively, evidentially verified by any source available to this audit — this should be stated as a real gap in the manuscript, not glossed over.
Known limitations	`admin/src/lib/permissions.ts` is a UI convenience only — if it and an RLS policy ever drift apart, the practical effect is either a hidden-but-still-reachable feature (if RLS is stricter than the UI assumes — safe but confusing) or, more seriously, a UI element that appears available but is correctly rejected by RLS (safe, but a poor user experience) — neither drift direction has been systematically checked across all 8 roles.
Additional testing required	A scripted (not manual) regression suite — exactly what the 2026-09 report's own §7 recommends — covering every role × every sensitive table, run against disposable test accounts. This is the single highest-value piece of missing evidence for the manuscript's Chapter 3 security claims.

---

3. Abuse Incident Reporting (Survivor Mobile)

Field	Detail
Intended user/role	Survivor
Purpose	Let a survivor submit a formal incident report, optionally on behalf of a third party, with a location and category, and receive a case number for tracking
Mobile screen	`mobile/src/screens/incident/ReportIncidentScreen.tsx`
Source files	Same file for form + submission logic; `mobile/src/lib/audit.ts` (`generateCaseNumber` — **client-side** case-number generation, confirmed by direct read; the 2026-09 report independently documents the same design choice)
Backend	`case_reports` table (insert), `case_status_history` (a seed row — "Report submitted" — inserted immediately after, so Case Tracking has something to show); RLS requires `identity_verification_status = 'verified'` before a report can be submitted — enforced both client-side (pre-check, for a clear message) and at the database policy level (the actual gate)
Workflow	Fill form (case type from a real 7-value enum: Physical/Psychological/Sexual/Economic Abuse, Stalking, Neglect, Others — confirmed against `supabase/module20_government_reporting.sql` 's `alter type case_type add value 'Stalking'` and the base schema's other 6 values) → choose incident location

	(registered address / current GPS / pick-on-map) → submit → `case_reports` insert → case number returned and shown
Required permissions	Authenticated survivor, `identity_verification_status = 'verified'`
Validation / error handling	Required-field checks client-side; **hardened this working session** — the submission call was previously missing a try/catch, meaning a genuine network failure (not just a returned database error) could leave the Submit button permanently stuck in a loading state with no feedback. Fixed to show an explicit "has NOT been submitted, please check your connection" message instead **[SESSION — this is a fix made earlier in this same session, confirmed by `tsc` / `expo-doctor` /bundle-export after the change, not by a live device test]**.
Existing test evidence	**None found as an executed test** — the fix above was verified for *build correctness* (it compiles, the app still bundles), not for *runtime behavior on a real device under a real network failure*.
Implementation status	**Implemented in code, integrated with the backend.** Not confirmed "tested successfully" in the sense of an observed, evidenced successful submission against the live project.
Known limitations	**Resolved (was a [GAP] in the first pass):** a survivor **cannot** attach evidence or a photo to an incident report from the mobile app. Confirmed by a full-text search of `mobile/src` for any reference to the `evidence` table or the `evidence` storage bucket — there are **zero** matches anywhere in the mobile codebase. Evidence upload **is** implemented, but only on the admin web dashboard — `admin/src/pages/cases/CaseDetailPage.tsx` (`submitEvidenceUpload` , lines ~622–651) lets a **staff member** upload a file against an existing case (private `evidence` bucket, path `{case_id}/{timestamp}_{filename}` , row recorded in the `evidence` table with `uploaded_by` , viewed later via a 60-second signed URL — `viewEvidence` , line 653 — not a long-lived public link) with a matching `logAudit()` call. So the capability exists in the system as a whole, but it is **staff-mediated**, not survivor-initiated from the reporting flow itself **[CODE, confirmed 2026-10-13]**.
Additional testing required	A real submission against the live project with a disposable test account, confirming the case number format, the `case_status_history` seed row, and — specifically — an airplane-mode test to confirm the network-failure fix actually behaves as intended on a physical device (this audit could only confirm it *compiles*, not that it *behaves correctly at runtime*).

4. Emergency SOS

Field	Detail
Intended user/role	Survivor (trigger), Municipal Admin / Provincial Admin / Provincial Staff / Police Officer (authorized recipients)
Purpose	Let a survivor in immediate danger send their location to authorized responders with minimal friction, while making failure states honest rather than silently swallowed
Mobile screens	`mobile/src/screens/sos/SosScreen.tsx` (status display, live map), `mobile/src/screens/home/SosConfirmSheet.tsx` (confirm → 5-second cancellable countdown → activation)
Source files	`mobile/src/lib/useSosTrigger.ts` (the actual state machine both screens share — confirmed to be one shared implementation, not two parallel ones), `mobile/src/lib/sosLocation.ts`
Backend	`sos_alerts` table (insert: `profile_id`, `latitude` / `longitude` / `accuracy` / `source`, `status: 'Triggered'`); recipient fan-out via `notify_on_sos_created()` trigger (latest definition in `module31_barangay_action_requests.sql`) to active `provincial_admin` / `super_admin` and the survivor's own `municipal_admin` — confirmed by direct read of the trigger body, which writes a deliberately generic notification title/message ("SafeHer Emergency Alert" / "An emergency alert requires your attention.") with **no location, name, or case detail** — relevant to the "must not expose sensitive information in notification previews" requirement **[SCHEMA, read directly]**
Workflow	Tap SOS → confirm → 5-second cancellable countdown → location preflight (permission/services check, fails fast with a specific reason if denied/off) → GPS fix attempt → insert `sos_alerts` row → realtime-subscribed `SosScreen` shows live status as an admin updates it (`Triggered → Responding → Resolved`)
Required permissions	Authenticated survivor; location permission is requested but **not required** — `sendWithoutLocation` is an explicit, offered fallback path, confirmed in code, not inferred
Validation / error handling	A synchronous `busyRef` (not just React state, which updates asynchronously) prevents a double-tap from creating two alerts — confirmed by direct read. **Hardened this working session**: (1) the alert-save call previously lacked a try/catch, meaning a true network failure could leave the UI stuck on "Sending your alert…" forever with no error; (2) a separate bug meant a raw database error message (potentially naming internal tables/columns) could be shown directly to the user instead of a generic message. Both fixed; verified only by build/bundle checks, not a live device test **[SESSION]**.
Existing test evidence	None specific to SOS *creation* found as an executed test. The 2026-09 report's documented test run (§6) covers SOS **referral to police**, i.e. what happens after an alert already exists and is routed to an officer — not the creation step itself.

status	**Implemented in code, integrated with the backend.** The referral/assignment half has some documented prior test evidence (`[DOC-2026-09]`); the creation half (this table) does not.
Known limitations	**This audit found no evidence that SOS delivers an SMS, places a phone call, or sends an OS-level push notification as part of alert creation itself** — it creates a database row and relies on (a) the admin dashboard's own realtime subscription for admins actively viewing it, and (b) the separate, best-effort Expo push pipeline (`send-push-notification` Edge Function) for a device notification. **The manuscript must not claim SOS "calls police" or "sends an SMS" — no such integration exists in the code inspected.** No offline/low-connectivity fallback exists — if the device has no network at the moment of submission, the alert is not created, and (per this session's fix) the user is now told so explicitly rather than shown a false success. This matches the 2026-09 report's own recommendation #4 ("plan an offline/SMS fallback... scoped out deliberately rather than overlooked"), independently corroborated by this audit's own earlier finding in this same session that neither the SOS screen nor the incident-report screen had any offline-queueing capability.
Additional testing required	A real device test, specifically: (1) trigger SOS with location permission granted — confirm the alert appears in the admin dashboard in real time; (2) trigger with location denied — confirm the "send without location" path still creates a record; (3) trigger in airplane mode — confirm the new "not sent" messaging actually appears rather than a stuck spinner. None of these three have been observed by this audit; all three are plausible-but-unconfirmed based on source review alone.

5. Case Assignment, Status Transitions, and Escalation

Field	Detail
Intended user/role	`barangay_admin` , `municipal_admin` , `provincial_staff` — route-gated `requireRole={['municipal_admin','provincial_staff','barangay_admin']}` (`admin/src/App.tsx:72-86`). `provincial_admin` / `super_admin` do not reach this route in the current RBAC split, even though their underlying table access is broader **[CODE]**.
Purpose	Single-screen case lifecycle management: status updates, staff assignment, shelter referral, police-officer referral, and barangay→municipal→provincial ownership escalation.
Web pages	`admin/src/pages/cases/CaseListPage.tsx` , `admin/src/pages/cases/CaseDetailPage.tsx` (1468 lines, read in full this pass)
Back end	Tables: `case_reports` , `case_status_history` , `case_assignments` , `case_referrals` , `shelter_referrals` , `law_enforcement_referrals` . Functions: `admin_can_access_case_active()` , `can_refer_case()` , `apply_case_referral()` trigger, `least_loaded_law_enforcement_officer()` RPC **[SCHEMA]** (`module2_case_data_rls.sql` , `module3_barangay_hierarchy.sql` , `module9_law_enforcement.sql` , `module26_law_enforcement_auto_assignment.sql`).
Wo-	

rkflow	**Status update** (`submitStatusChange` , `:240-279`): updates `case_reports.status` , inserts a `case_status_history` row, notifies the survivor; a DB trigger audits it automatically. **Assignment** (`submitAssignment` , `:281-323`): inserts `case_assignments` ; auto-advances `Under Review` → `Assigned` . **Shelter referral** (`submitReferral` , `:325-373`): inserts `shelter_referrals` , forces `case_reports.status='Referred'` , calls `logAudit()` manually (no DB trigger on this table). **Police referral** (`submitLawEnforcementReferral` , `:588-620`): auto-assigns via RPC `least_loaded_law_enforcement_officer()` , with a manual "assign officer" fallback if none is eligible. **Escalation** (`submitEscalate` , `:380-402`): inserts into `case_referrals` (`status:'Completed'` hard-coded — no accept/reject step exists yet); a DB trigger (`apply_case_referral()` , module3) is the *only* code path that ever writes `case_reports.owner_level` / `owner_barangay_id` — the client never updates those columns directly **[CODE + SCHEMA]**.
Required permissions	No component-level role conditional gates the Update Status / Refer to Shelter / Refer to Police Officer buttons — any role reaching the route sees them; the real boundary is RLS, not the UI (confirmed by an inline code comment in the file itself calling this "a convenience gate, not the real boundary"). `case_reports` UPDATE policy = `admin_can_access_case_active(case_id)` (province-wide for provincial tier, municipality-match for municipal, barangay-owner-match for barangay — **not** scoped by current `owner_level` in the UI, only in the RLS policy). `case_referrals` INSERT independently re-derives the caller's authority server-side via `can_refer_case()` , not merely trusting the UI's own escalation-eligibility check **[SCHEMA]**.
Validation / error handling	Status-sequence validation (`CASE_STATUS_FLOW` , Pending→Under Review→{Assigned,Referred}→…→Closed) is **UI-only** — a repository-wide grep of all 32 SQL modules found no trigger/CHECK constraint enforcing status sequence at the database layer, unlike the comparable transition guards that do exist for `expert_support_requests` , `sos_alerts` , and `barangay_action_requests` . This is a genuine gap: a direct API call bypassing the admin UI could set any status to any other status, limited only by RLS ownership, not sequence **[SCHEMA, GAP]**. No `try/catch` exists anywhere in `CaseDetailPage.tsx` / `CaseListPage.tsx` ; `submitStatusChange` / `submitAssignment` perform no error check on their Supabase calls at all (a failed/RLS-denied write silently closes the modal), while `submitEscalate` / `submitLawEnforcementReferral` do check `{ error }` but only log to console, never surface a toast **[CODE]**.
Existing test evidence	None found — repository-wide search for `*.test.*` / `*.spec.*` returns zero results outside `node_modules` **[CODE, absence confirmed]**.
Implementation status	**Partially implemented.** Core workflows are implemented in code and wired to real tables/RLS, internally consistent between UI and policy in every branch checked, but status-transition integrity relies entirely on client-side enforcement with no DB-level backstop, and several write paths have no user-facing error feedback. No runtime test evidence exists for any of it.
Known limitations	Status-flow rules are TS-only, not Postgres-enforced. `submitStatusChange` / `submitAssignment` never surface a write failure to the admin. Escalation is a Phase-1 "immediate transfer" with no accept/reject step (`responded_by` / `responded_at` columns exist in the schema but are unused — reserved for a future phase). A `barangay_admin` who escalates a case away loses `case_reports` SELECT access to it entirely (a documented Phase-1 limitation in the schema comments, not a bug). Action buttons are not hidden based on current case ownership — RLS is the real gate, discovered only when a write is attempted, not before.

testing required	Confirm a raw/non-UI status write for a disallowed sequence actually succeeds (would prove the DB-level gap); confirm RLS denial on out-of-scope role writes to `case_reports` / `case_assignments` / `case_referrals`; confirm escalation only ever changes `owner_level` / `owner_barangay_id` via the trigger, never a direct update; confirm notification delivery on each write path; check for a race condition on concurrent case assignment (no uniqueness guard found on `case_assignments`).

6. Shelter Requests and Shelter Administration

Field	Detail
Intended user/role	`shelter_admin` for `/admin/shelter/information`, `/admin/shelter/occupancy`, `/admin/shelter/requests` (own shelter only); `municipal_admin` / `provincial_staff` for `ShelterManagementPage` / `ShelterReferralPage` (the referring-agency side, not the shelter's own side) **[CODE]**.
Purpose	Let agency staff refer a case to a shelter and manage the province/municipality shelter directory; let a shelter's own admin manage its info, capacity/occupancy, and incoming requests/referrals.
Web pages	`ShelterInformationPage.tsx`, `ShelterManagementPage.tsx`, `ShelterOccupancyPage.tsx`, `ShelterReferralPage.tsx`, `ShelterRequestsReferralsPage.tsx`, `ShelterAdminDashboard.tsx`, plus the shared `admin/src/hooks/useShelterAdminData.ts`
Back end	Tables: `shelters`, `shelter_requests`, `shelter_referrals`. Functions: `current_shelter_id()`, `is_shelter_admin()`, `validate_shelter_update()` trigger, `validate_shelter_request_update()` trigger **[SCHEMA]** (`module4_shelter_admin.sql`, `module7_shelter_staff.sql`, `module24_retire_shelter_staff_role.sql`).
Workflow	`shelter_requests` (survivor-initiated, mobile, no case required) and `shelter_referrals` (agency-initiated, always tied to a `case_id`) are **two deliberately separate tables, confirmed by an explicit "ARCHITECTURE NOTE" in `module4_shelter_admin.sql`** — never merged in the schema, though the admin UI *presents* them together in one unified pipeline table via a discriminated union (`kind:'request'
Required permissions	RLS scopes `shelter_admin` strictly to their own shelter via `current_shelter_id()` on `shelters` / `shelter_requests` / `shelter_referrals`; a trigger additionally blocks a `shelter_admin` from editing their own shelter's name/address/coordinates even though they can edit occupancy/capacity, matching the UI's own read-only treatment of those fields. All queries are also filtered client-side by `profile.shelter_id` as a second (non-authoritative) layer **[SCHEMA + CODE]**.
Validation / error handling	No `try/catch` anywhere across the five pages or the shared hook. Three of the five pages (`ShelterInformationPage`, `ShelterManagementPage`, `ShelterOccupancyPage`) do check `{ error }` on their own direct calls and show a toast on failure. By contrast, the **shared hook's** `updateRequestStatus` / `updateReferralStatus` / `bumpOccupancy` — used by `ShelterRequestsReferralsPage` and `ShelterAdminDashboard` — never check the `{ error }` result of their own Supabase calls at all; they unconditionally show a success toast regardless of whether the underlying write actually succeeded. This is a concrete gap: a silently-failed status change or occupancy bump would be reported to the admin as successful **[CODE]**. `ShelterManagementPage` does validate capacity (`capacity >= occupancy`, non-negative) and coordinates client-side before submit — the only real input validation found in this area.

evidence	None found, same repository-wide search as above **[CODE, absence confirmed]**.
Implementation status	**Partially implemented.** All five pages are implemented and wired to live tables/RLS with role scoping that matches the database layer, but shelter capacity/occupancy tracking is a fully manual, unguarded integer with no DB-level consistency check, and the shared mutation hook silently ignores write failures.
Known limitations	`current_occupancy` is a plain integer column with **zero DB-level automation** — no trigger, generated column, or CHECK ties it to actual accepted requests/referrals; it is bumped only by application code in three independent places (shelter referral acceptance, shelter request confirmation, and a free-form "edit capacity" modal), and there is no "discharge" workflow that ever decrements it — occupancy can drift from reality, and this is explicitly acknowledged in the code's own comments, not something this audit is the first to notice. `shelter_staff` is a **retired-but-intentionally-not-removed** role (`module24_retire_shelter_staff_role.sql` migrated existing accounts to `shelter_admin` and blocks new `shelter_staff` creation) — the enum value, helper functions, and UI conditionals referencing it remain throughout `admin/src` by explicit design ("an intentionally non-destructive retirement," per the module's own header), not as accidental dead code. `ShelterAdminDashboard.tsx` links to `/admin/shelter/services`, which does not appear as a registered route anywhere in `App.tsx` — this looks like a leftover link from an earlier page consolidation, but could not be confirmed as broken without running the app **[GAP]**.
Additional testing required	Runtime confirmation of whether the `/admin/shelter/services` link actually 404s; a concurrency test on `bumpOccupancy` (read-then-write, no atomic increment or row lock — two simultaneous acceptances could race); confirmation that a `shelter_admin` truly cannot read/write another shelter's rows at the RLS layer, not just that the client hides it; confirmation (via actual execution, not static reading) that a failed status-update write is surfaced to the admin in some way the code itself doesn't show.

7. Police Officer Referrals and Access Restrictions

Field	Detail
Intended user/role	`law_enforcement` ("Police Officer" in the UI)
Purpose	Direct province-wide SOS emergency response, plus narrower case-referral-based review of specific cases referred by case-management admins — **two distinct access models that this audit found are no longer the same mechanism** (see "Known limitations").
Web pages	`SosReferralsPage.tsx`, `CaseReferralsPage.tsx`, `LawEnforcementDashboard.tsx`, `PoliceManagementPage.tsx`, plus the referral-creation UI inside `CaseDetailPage.tsx`

Workflow	**SOS alerts**: every active `law_enforcement` account has **direct, unconditional, province-wide** SELECT+UPDATE on `sos_alerts` via the policy `"Police officers can view all sos alerts"` (`using (current_role() = 'law_enforcement')`) — no referral row or assignment is involved at all for this path. The officer self-acknowledges and moves status forward-only Triggered→Responding→Resolved, enforced by a trigger **[SCHEMA]** (`module19_police_direct_sos_response.sql`). **Case referrals**: a case-owning admin clicks "Refer to Police Officer" in `CaseDetailPage.tsx` ; the system auto-assigns the least-loaded active officer via RPC, or leaves it unassigned with a manual-assign fallback; the referred officer only gains visibility into that specific case once `assigned_officer_id` is set — this path **is still assignment-gated**, unlike SOS **[CODE + SCHEMA]**.
Required permissions	`admin/src/lib/permissions.ts` grants `law_enforcement` only a `reports.view` key — the real dashboards are reached via dedicated routes outside the generic permission system. The actual enforcement is RLS: unconditional province-wide access for SOS, `assigned_officer_id = current_profile_id()` for case referrals. Referral creation is restricted to case-owning admin tiers only — `law_enforcement` cannot create or reassign its own referrals (a trigger blocks self-reassignment) **[SCHEMA]**.
Validation / error handling	Both referral pages show an error banner with retry on load failure and a toast on action failure; `CaseDetailPage.submitLawEnforcementReferral` only logs referral-creation errors to the console, with no user-facing toast — a small, verifiable inconsistency with its sibling pages **[CODE]**. Forward-only SOS status transitions and self-only acknowledgment are enforced server-side, not just in the UI.
Existing test evidence	None found as an executed/automated test. Each SQL module contains an extensive hand-run "VERIFICATION" comment block (SQL queries meant for a human to run against a live project) — these are instructions, not evidence that they were actually run **[CODE, absence of automated evidence confirmed]**.
Implementation status	**Implemented in code, integrated with the backend.** No automated or documented execution evidence exists, so "tested successfully" cannot be claimed for either access path.
Known limitations	**No jurisdiction/station/precinct concept exists anywhere in the schema** — SOS access is explicitly province-wide for every active officer; an officer in one municipality can see and act on an alert triggered anywhere in the province. This is a **material change from the officer-assignment model that `SafeHer_System_Documentation_Report.html` §04–06 describes and demonstrates with a worked diagram and a documented live-RLS test run** ("an alert is only ever visible to the specific officer it is assigned to, never a shared queue") — `module19_police_direct_sos_response.sql` 's own header states this direct-access model was adopted specifically to reduce emergency-response latency, after confirming no station/jurisdiction concept exists in the province's own structure to assign by. **This means the prior report's §06 test evidence (8 PASS scenarios) now documents a mechanism the SOS pathway no longer uses** — it remains accurate for the case-referral pathway only, which is still assignment-gated. See `03-prior-documentation-reconciliation.md` for the full comparison. Auto-assignment for case referrals has no geographic/specialty weighting, only least-loaded.

Additional testing required	Live RLS verification of the current (module19) direct-access model specifically — the 2026-09 test run verified the superseded model, not this one; a fresh live test confirming an officer in Municipality A can/cannot see an SOS alert from Municipality B would directly settle whether the "no jurisdiction restriction" finding is also true in the deployed project, not just in the migration file.

8. AI Support, Safety Triage, and Human-Support Handoff

Field	Detail
Intended user/role	Survivor (mobile, chat itself); `support_expert` (human handoff conversations); `provincial_staff` / `provincial_admin` / `super_admin` (queue assignment)
Purpose	Tiered AI/rule-based support chat ("SafeSpace Assistant") with safety-keyword detection and a survivor-initiated, opt-in handoff to a human expert.
Web pages / mobile screens	`mobile/src/screens/assistant/SafeHerAssistantScreen.tsx` , `mobile/src/lib/useSupportChat.ts` ; admin: `SupportQueuePage.tsx` , `SupportExpertDashboard.tsx`
Backend	Tables: `chat_sessions` , `chat_messages` , `expert_support_requests` , `faqs` . Edge Function `safeher-chatbot` , calling Groq's OpenAI-compatible API (model `openai/gpt-oss-120b`) via a server-side-only `AI_API_KEY` **[SCHEMA + CODE]** (`module6_ai_support.sql` , `supabase/functions/safeher-chatbot/index.ts` , `providers/groq.ts`).
Workflow	A survivor's message is inserted client-side (RLS-governed, self-only). If the session is in `AI` mode, the Edge Function runs, in strict order, stopping at the first match: (a) a **deterministic regex safety check** with no network call — an EMERGENCY/URGENT match short-circuits straight to a safety panel (SOS/Expert/Shelter action buttons), skipping both the FAQ layer and the AI call entirely; (b) an FAQ exact/keyword match against admin-authored content; (c) a deterministic guided-intent keyword match; (d) the Groq AI call as the last resort, required to return strict JSON, parsed defensively, with the AI's own risk assessment only ever allowed to escalate — never downgrade — the rule engine's result. The AI/System reply is inserted using a service-role client; RLS explicitly blocks any authenticated client from inserting a message with `sender_type` of `'AI'` or `'System'` . **Human handoff is survivor-initiated only** — nothing in the Edge Function or any trigger automatically creates an `expert_support_requests` row on an EMERGENCY/URGENT detection; the survivor must tap "Talk to an Expert." A provincial-tier admin then manually assigns a `support_expert` , which flips the session to `HUMAN` mode and pauses the AI (the Edge Function returns 409 if called while in `HUMAN` mode) **[CODE + SCHEMA]**.

permissions	A survivor sees only their own chat session; `'Expert'`-sender messages are self-only and only while that session is actively assigned to that expert. `expert_support_requests` assignment is restricted to `is_provincial_admin()` (covers `provincial_admin` / `super_admin` / `provincial_staff`) or the already-assigned expert. **Notably, this same `is_provincial_admin()` branch also appears in the `chat_sessions` / `chat_messages` SELECT policy** — meaning those three roles can read **any** survivor's full chat transcript at the database level, not only an assigned `support_expert` — though no dedicated admin UI page was found that exposes this browsing capability; it is a real but UI-unexercised DB-level capability, not something independently confirmed as ever used **[SCHEMA]**.
Validation / error handling	The Edge Function requires an `Authorization` header, validates the caller's profile, validates input presence/type, and caps message length at 2000 characters. Any unhandled exception or a Groq-call failure degrades to a generic "unavailable" response rather than a stack trace, with raw provider error text surfaced only behind an explicit debug flag gated to dev builds. AI output is never trusted as-is — `reply` / `risk_level` / `needs_human_support` / `actions` are all validated/clamped server-side before being stored or returned. The mobile client wraps both the DB insert and the function call in try/catch, sets a user-facing fallback message pointing to SOS/Shelter/Expert rather than leaving the UI hanging, and shows an explicit offline banner via network state with input disabled while offline **[CODE]**.
Existing test evidence	None found as an automated test.
Implementation status	**Implemented in code, integrated with the backend** for the safety triage and opt-in human handoff. **Automatic crisis→human escalation does not exist** — only UI prompts are shown on a safety-flagged message; if the manuscript were to claim automatic escalation on crisis detection, that would be inaccurate — the real mechanism is informational/opt-in, confirmed by direct code read, not inferred.
Known limitations	The project's own `README.md` claim of `ANTHROPIC_API_KEY` as the AI provider is **stale/incorrect** — the Edge Function as currently written only reads `AI_API_KEY` and calls Groq, never Anthropic (this discrepancy was already flagged in `00-audit-scope-and-access.md`, and is independently reconfirmed here by a direct read of the current Edge Function code). The AI key is read exclusively server-side via `Deno.env.get()` and never returned to or called directly by the client — **no key-exposure issue was found**. Safety detection is a fixed English-language regex list — no Tagalog/Filipino-language patterns, no NLP/ML classifier — conservative by design but will miss non-matching phrasings. Crisis detection never auto-triggers SOS or auto-creates an expert request.
Additional testing required	Live verification of the chat RLS policies (cross-survivor isolation, self-assignment rejection); confirmation that the AI provider secret is actually present and valid in the deployed project (code supports its graceful absence, but presence was not verified here); manual QA of the offline/timeout/rate-limit path on a real device.

9. Notifications and Delivery Mechanisms

Field	Detail
Intended user/role	Survivors (mobile inbox) and all staff/admin roles (web inbox) — each sees only notifications addressed to their own `profile_id`.
Purpose	Deliver an in-app notification record for case/SOS/referral/assignment/support/system/announcement events, with a best-effort Expo push alert layered on top for a subset of high-priority types.
Web pages / mobile screens	`mobile/src/lib/pushNotifications.ts`, `mobile/src/screens/notifications/NotificationsScreen.tsx`, `admin/src/pages/notifications/NotificationsPage.tsx`, Edge Function `send-push-notification`
Back end	Tables: `notifications` (base schema, extended by module8 with push-delivery-status columns), `push_tokens` (module8). Functions: `dispatch_push_for_notification()` trigger, plus per-module notify functions across modules 6, 7, 9, 10, 14, 17, 19, 28, 31 **[SCHEMA]**.
Workflow	An event (a client-side insert into `notifications`, e.g. from `CaseDetailPage.tsx`'s status/assignment/referral actions, or a server-side trigger, e.g. SOS creation, identity-verification decision, provincial announcement fan-out) inserts a `notifications` row. A trigger checks the notification type against an allow-list (`SOS, Case Update, Referral, Assignment, Support` — **not** `Announcement` or `System`, which stay in-app only) and, if configured, calls the `send-push-notification` Edge Function, which re-reads the notification with the service-role key, looks up the recipient's active Expo push tokens, sends to Expo's push API with genericized copy, deactivates tokens Expo reports as unregistered, and records a delivery status. Client screens read `notifications` filtered to their own `profile_id`; the mobile screen additionally holds a Realtime subscription for live updates, while the admin page only refreshes manually **[CODE + SCHEMA]**.
Required permissions	`push_tokens` RLS restricts every operation to the token owner — no role, including `provincial_admin`, can read another profile's tokens. `notifications` access is scoped to the recipient; the UPDATE grant is limited to the `is_read` column only. The `send-push-notification` function has no end-user caller at all — it is invoked only by the DB trigger, authenticated via a shared internal secret header, not a user JWT **[SCHEMA + CODE]**.
Validation / error handling	The client explicitly detects and refuses to run inside Expo Go (since the push module crashes synchronously there on Android) and returns a typed result for each failure mode (permission denied, no EAS project configured, storage failure). **Per the client file's own header comment, no EAS project has been configured for this app yet** — meaning `getExpoPushTokenAsync()` cannot currently obtain a real token in this project's state. The Edge Function validates its secret header and payload, records `'no_recipients'` rather than failing outright when nobody has a token, wraps the Expo API call in try/catch, and always writes a dispatch status back regardless of outcome. The database trigger wraps its own dispatch call in an exception handler so a push-dispatch failure can never roll back the authoritative notification row — it only raises a

	warning. The mobile notifications screen rolls back an optimistic "mark read" on failure; the admin notifications page does not, so a silent failure there leaves the UI believing an item was marked read when it was not **[CODE]**.
Existing test evidence	None found as an automated or executed test.
Implementation status	**Partially implemented.** The in-app pipeline (trigger → table → list screens, with Realtime on mobile) is implemented and wired to real tables/RLS. Push delivery is fully implemented in code (Edge Function, Expo API call, token-deactivation bookkeeping) but, per the code's own stated configuration state, **is not currently operable end-to-end** because no EAS project is configured — there is no code-level evidence any real device has ever received a push notification from this system.
Known limitations	No EAS project configured → remote push cannot currently reach a real device. `Announcement` / `System` notifications never trigger push by design, in-app only. The admin notifications page has no Realtime subscription and no rollback on a failed mark-read. Delivery confirmation only goes as far as Expo accepting the push *ticket* — the Edge Function does not poll Expo's separate push *receipt* API, so a "sent" status does not prove the device actually displayed the notification. A one-time operator SQL configuration step (populating the internal dispatch URL/secret) is documented as manual, with no evidence in the repository that it has been run on the live project.
Additional testing required	An end-to-end push test on a real EAS development build once an EAS project exists; verification that the operator-side dispatch configuration has actually been applied on the live project; a check against Expo's receipt API to confirm true device delivery, not just ticket acceptance; manual QA of the admin notifications page's error paths.

10. Reports, Analytics, Audit Logs, Backups, and Recovery

Field	Detail
Intended user/role	Audit Log and Backup/Restore: `provincial_admin` / `super_admin` only. Reports/Analytics: all staff/admin roles, geographically scoped by role tier, with government-template (WEDC/CNSP/CICL) reports further restricted to tiers with survivor-PII access.
Purpose	Audit Log: a province-wide activity trail of administrator/case/access/backup actions. Backup/Restore: export and controlled recovery of **administrative reference data only** — not a full database backup (see "Known limitations"). Reports/Analytics: real aggregated case/referral/SOS statistics and government-template PDF reports generated from live data, not placeholder data.
Web pages	`AuditLogPage.tsx` , `BackupRestorePage.tsx` , `ReportsPage.tsx` , `AnalyticsPage.tsx` , plus role-specific variants (e.g. `PoliceReportsPage` , `ShelterReportsPage` , `SupportReportsPage`)
Back end	Tables: `audit_logs` , `system_backups` , `backup_schedule` , `system_restores` . Functions: generic `audit_row_change()` trigger (attached only to a specific subset of tables — see limitations), `set_backup_schedule()` / `begin_restore_operation()` RPCs. Edge Functions: `backup-create` , `backup-restore` . Storage bucket: `system-backups` (private, 60-second signed URLs only) **[SCHEMA]** (`module16_technical_admin_backup.sql` , `module23_backup_restore.sql` ,

	`module11_provincial_staff_rbac.sql`).
Workflow	**Audit**: any `logAudit()` call (31 call sites found across 17 admin files, covering case status/assignment changes, user/account management, report/PDF generation, and backup/restore operations) or the generic DB trigger inserts into `audit_logs` ; the page lists it with real, computed-from-actual-data filters, not an invented action list. **Backup create**: the Edge Function exports only `municipalities` , `barangays` , `shelters` , and non-survivor `user_profiles` rows, gzips, SHA-256-checksums, uploads to the private bucket, and records a row with an audit entry; a validation routine separately checks checksum/row-count/schema-version/expiry before any restore is allowed to proceed. **Restore**: a three-step wizard (begin → confirm → execute) that re-validates the backup at every step, takes a server-side advisory lock, requires a server-generated confirmation phrase to be typed back, creates a **mandatory pre-restore safety backup** (aborting the whole restore if that fails), upserts rows by primary key (**never deletes**, specifically to avoid orphaning live foreign-key-dependent rows), explicitly excludes administrator-account fields from the restore payload (to avoid silently reinstating revoked access), and runs a post-restore read-only health check before reporting success. **Reports/Analytics**: a shared controller fetches live, role/filter-scoped case/referral/SOS rows; every chart and table is sourced from a real Supabase query, not static/placeholder data, confirmed by direct read **[CODE + SCHEMA]**.
Required permissions	`audit_logs` SELECT RLS is restricted to `is_provincial_technical_admin()` — `provincial_staff` and every other role is excluded, matching the page's own UI copy. Both backup/restore Edge Functions independently re-check the caller's role server-side (not just trusting the UI), and no authenticated role has any direct INSERT/UPDATE/DELETE grant on `audit_logs` / `system_backups` / `system_restores` at all — only the service role or the two SECURITY DEFINER RPCs can write **[SCHEMA + CODE]**.
Validation / error handling	The audit log page's own UI copy discloses directly, in the product itself, that the log is **"not append-only/immutable at the database level today — treat it as an activity record, not a tamper-evident compliance log"** — this is a self-reported limitation in the application, not something inferred by this audit. The backup/restore path is unusually carefully layered (checksum + row-count + schema/project checks at validate time, re-validated again immediately before execution, an atomic compare-and-swap to prevent double-execution from a repeated click, a mandatory safety backup before any destructive step) — but all of this is still **unexecuted code** as far as this repository shows; see "Existing test evidence" **[CODE]**.
Existing test evidence	None found anywhere in the repository for any of audit logging, backup, restore, reports, or analytics — no automated test files exist, and no log or record in the repository demonstrates a backup or restore has ever actually been executed against a populated database **[CODE, absence confirmed]**.
Implementation status	Audit Log and Reports/Analytics: **Implemented in code, integrated with the backend** — no placeholder/static data was found anywhere in the reporting pages. Backup/Restore: **Implemented in code, integrated with the backend**, but explicitly **not verified** — the code's own careful design (safety backup, checksum re-validation, upsert-only scope) is evidence of careful engineering, not evidence that a restore has ever successfully run; claiming "tested successfully" would overstate what the code alone shows.

limitations	**Backup/restore covers only administrative reference data** (municipalities, barangays, shelters, non-survivor admin accounts) — explicitly **not** survivor records, case data, evidence, credentials, audit logs, or notifications. **Supabase point-in-time recovery (PITR) — the only mechanism that would actually protect case/survivor/SOS data — is confirmed disabled on this project** (documented in the module's own comment block, sourced from a one-time `supabase backups list` check showing `{"pitr_enabled":false,"backups":[]}`); this means there is currently **no full-database recovery mechanism of any kind** for survivor-facing data, only for the administrative reference tables. This directly reaffirms — rather than resolves — the 2026-09 report's own §07 recommendation #1 to enable PITR, which per this evidence has **still not been acted on**. The generic audit trigger covers only a specific subset of tables by deliberate design (excluding `evidence`, `shelter_referrals`, `notifications`, `sos_alerts`, which instead rely on developers remembering to call `logAudit()` manually) — audit-log completeness therefore depends on every future mutating feature remembering to add that call; this audit identified 31 existing call sites but did not attempt a full line-by-line completeness audit of every mutation in the codebase, so audit-log coverage should be described as "substantial but not exhaustively verified," not "complete."
Additional testing required	A real backup-create → validate → restore (begin/confirm/execute) run against a staging database, including a deliberately corrupted or expired backup to confirm validation actually blocks a restore; a concurrency test of two admins attempting a restore simultaneously; a completeness audit of every mutating action in the codebase against `logAudit()` call sites to state audit-log coverage precisely rather than approximately; confirmation of the live project's current PITR status directly (this audit's finding is sourced from a code comment describing a past check, not a live check performed in this pass).

Unresolved questions / structural and documentation gaps

The per-feature "Additional testing required" rows above are the authoritative, feature-specific list of outstanding runtime verification — they are not repeated here. This section covers the broader, cross-cutting questions the governing master prompt asked this pass to resolve or explicitly report as still open.

1. **Resolved this pass**: whether evidence/photo attachment exists on the mobile incident-report flow — it does not; see section 3's "Known limitations" row.
2. **Resolved this pass**: whether the `avatars` storage bucket is SQL-migrated — it is not, confirmed by the project's own code comment; see `01-deliverable-1-project-structure.md`.
3. **Still unresolved, unchanged from the first pass**: whether all 32 migration modules — and the live RLS policies they define — have actually been applied to the live Supabase project exactly as written is unverifiable from the repository alone; this audit still has no live, read-only database credential or connection string in this environment. This is the single largest remaining evidence gap for the manuscript's Chapter 3 security claims, since every RLS conclusion in this document (including the new sections above) is sourced from migration **files**, not a live schema/policy introspection.
4. **Newly identified this pass**: `SafeHer_System_Documentation_Report.html` §04–§06 describes and demonstrates (with a worked diagram and a documented 8-scenario live-RLS test run, all PASS) an officer-assignment-gated model for SOS visibility that `module19_police_direct_sos_response.sql` has since **replaced** with direct, unconditional, province-wide SOS access for every active `law_enforcement` account. The prior report's test evidence is therefore now evidence for a

superseded mechanism, not the one currently in the code, for the SOS pathway specifically (it remains accurate for the case-referral pathway, which did not change). See `03-prior-documentation-reconciliation.md` for the full comparison and why this matters for how the manuscript should cite that prior test run.

5. **Newly identified this pass**: `SafeHer-System-Inventory.pdf` remains unreadable in this environment (no PDF rendering tool available) — its contents are still not reflected anywhere in this audit, unchanged from the first pass.

Next files/evidence this audit would need to continue: live (read-only) access to the Supabase project for a schema/policy introspection remains the single highest-value missing input, for the reasons in item 3 above; the `SafeHer-System-Inventory.pdf` should be opened and reconciled by a team member with normal access; and the consolidated, prioritized list of what to verify next (across all 10 feature areas, not just this section) is in `05-unresolved-evidence-and-testing-register.md`.

SECTION 03

Reconciliation — SafeHer_System_Documentation_Report.html vs. Current SafeSpace Implementation

Status: Complete for §04–§07 (the sections the governing master prompt specifically flagged as unread); §01–§03 were read in full during the first audit pass and already summarized in 00-audit-scope-and-access.md, not re-litigated here
Source document date: 2026-10-13
Source file: `docs/capstone-evidence/03-prior-documentation-reconciliation.md`

Purpose and scope

`00-audit-scope-and-access.md` (first pass) established that `SafeHer_System_Documentation_Report.html` is a genuine, team-authored prior artifact (its author list matches this repository's git history) and read it in full, concluding its *architecture and methodology* descriptions remain substantively accurate while its *counts* (roles, modules, Edge Functions) are stale. This document goes one level deeper: it reads §04 ("System Features") and §05 ("System Demonstration") — which that first pass had only seen via section headers, not full content — in full, alongside §06 ("Testing & Evaluation") and §07 ("Conclusion & Recommendations"), and reconciles every specific, checkable claim in them against the current codebase. **No application code, schema, or configuration was changed to produce this document** — this is a read-only comparison.

Reconciliation table

#	Claim in the 2026-09 report	Section	Current status	Evidence
1	"26 survivor-facing mobile screens" (§04 table framing, "Mobile app — survivor-facing (26 screens)")	§04	**Not independently re-counted this pass.** The feature categories listed (onboarding/auth, SOS, incident reporting, case tracking, shelter, verification, AI assistant, case access/QR, profile/notifications) all still exist as named in the current `mobile/src/screens/` tree, confirmed by the Deliverable 1 structure walk in the first pass. Whether the exact screen *count* is still 26 was not re-verified in this pass and should not be assumed unchanged.	**[DOC-2026-09]** for the count; **[SESSION]** for the categories still existing
2	"9 roles, one codebase" / role table lists 8 rows (Barangay admin, Municipal admin, Provincial staff, Provincial admin/Super admin combined, Shelter admin, **Shelter staff**, Support expert, Law enforcement)	§04	**Stale.** `shelter_staff` is now a **retired** role — `module24_retire_shelter_staff_role.sql` migrated existing `shelter_staff` accounts to `shelter_admin` and blocks creation of new ones. The report describes it as a fully active, distinct responsibility ("Works cases assigned to them within one shelter; cannot self-assign") with no indication of retirement. 8 roles remain *active/assignable* today (survivor plus 7 staff roles), consistent with Deliverable 1.	**[SCHEMA]** `module24_retire_shelter_staff_role.sql`
3	"Law enforcement: Assigned referrals only… no shared queue (see §5)"	§04	**Partially stale.** True for case referrals (still assignment-gated, confirmed this pass). **False for SOS alerts** — see item 5 below, the report's central and most consequential stale claim.	**[CODE + SCHEMA]** this pass, section 7 of `02-deliverable-2-feature-matrix.md`
4	"Backend capabilities… 5 Edge Functions" (count established in §03, carried into Deliverable 1's stale-count note from the first pass)	§03 (not re-read this pass; cited from `00-audit-scope-and-access.md`)	**Stale.** 6 Edge Functions exist today: `create-admin-user`, `admin-user-metadata`, `safeher-chatbot`, `send-push-notification`, `backup-create`, `backup-restore`.	**[SESSION]**, confirmed again this pass via direct reads of `send-push-notification`, `backup-create`, `backup-restore`, `safeher-chatbot`
5	Worked example (§05) and its live-RLS test run (§06, 8 scenarios, all PASS): an SOS referral row with `assigned_officer_id` left null is **invisible to every officer** ("RLS: no officer visibility — dead end"); setting `assigned_officer_id` is what makes it **"visible to that officer only."** The diagram and its caption frame this assignment-gated model as the fixed, current design.	§05, §06	**Superseded for SOS alerts.** `module19_police_direct_sos_response.sql` replaced this model: every active `law_enforcement` account now has **direct, unconditional, province-wide** SELECT+UPDATE on `sos_alerts` — no referral row or `assigned_officer_id` is involved in SOS visibility at all anymore. The module's own header states this was a deliberate change to reduce emergency-response latency, made after confirming no station/jurisdiction/precinct concept exists in the schema or the province's own organizational structure to assign by instead. **This means the specific, documented 8-scenario PASS test run in §06 — real evidence that a real verification was once performed — now demonstrates a mechanism the SOS pathway no longer uses.** The case-referral pathway (a *separate* table, `law_enforcement_referrals`) is unaffected and remains assignment-gated exactly as described.	**[SCHEMA]** `module19_police_direct_sos_response.sql`, confirmed by direct read this pass

			reference data (municipalities, barangays, shelters, non-survivor accounts) — this is architecturally unchanged from what the report itself already disclosed as a limitation.	**[SCHEMA]**, confirmed this pass via direct read of `backup-create` / `back up-restore` and their migrations
7	§07 Recommendation #2: "Move live RLS verification into an automated suite… manual."	§07	**Not actioned.** Zero automated test files exist anywhere in the repository (`mobile/` , `admin/` , or `supabase/functions/`), confirmed repeatedly across this entire audit session via repository-wide search. Every module's own "VERIFICATION" block remains a block of SQL for a human to run by hand, not a script.	**[CODE]**, repeatedly confirmed, absence verified
8	§07 Recommendation #3: "Revisit the law-enforcement access model if the province formalizes a police organizational unit… Access is currently per-referral assignment rather than by precinct or unit, a deliberate choice made because no such organizational concept exists yet."	§07	**Revisited, but not in the direction the recommendation anticipated.** The recommendation assumed the model would stay assignment-based until a formal police organizational unit existed to assign by. Instead, `module19` broadened SOS access to be unconditional and province-wide — a different resolution to the same underlying problem (no station/precinct concept), not the precinct-scoped model the recommendation described. Worth citing precisely in the manuscript: the *gap* was real and acted on; the *solution chosen* was not the one originally proposed.	**[SCHEMA]**
9	§07 Recommendation #4: "Plan an offline/SMS fallback for low-connectivity areas… scoped out deliberately rather than overlooked."	§07	**Not actioned, confirmed still true.** This session's own earlier work (prior to this continuation) already confirmed neither the SOS screen nor the incident-report screen has any offline-queueing capability — consistent with `02-deliverable-2-feature-matrix.md` section 4's "Known limitations" row.	**[SESSION]**
10	§07 Recommendation #5: "Keep the single-admin-app decision, but watch its bundle size… the production build already warns on chunk size."	§07	**Not independently re-verified this pass** — would require actually running `vite build` and reading its warnings, which this pass did not do. Not assumed either way.	**[GAP]**
11	README.md: AI provider is `ANTHROPIC_API_KEY`	(README, not the HTML report)	**Stale/incorrect**, reconfirmed this pass by directly reading the current `safeher-chatbot` Edge Function: it reads only `AI_API_KEY` and calls Groq's API (`openai/gpt-oss-120b`), never Anthropic. Already flagged in the first pass (`00-audit-scope-and-access.md`); independently reconfirmed here against the actual current Edge Function source rather than taken on the first pass's word.	**[CODE]**, confirmed directly this pass
12	Report branding and framing throughout: "SafeHer," PSWDO Aurora Province footer, VAWC-focused language	Whole document	**Predates the SafeSpace rebrand and the master prompt's explicit gender-inclusive, multi-abuse-type scope statement.** This was already noted in the first pass. Reaffirmed here: no section of §04–§07 uses "SafeSpace" or gender-inclusive language; any manuscript text drawn from this report's prose must be rewritten for both, not merely copied with a find-and-replace of the name.	**[DOC-2026-09]**
13	§07 framing sentence: "SafeHer replaces a paper-and-phone-call coordination chain with one governed system of record spanning eight municipalities and nine distinct responder roles."	§07	Municipality count (8) not independently re-verified this pass — not contradicted by anything found, but also not re-confirmed against current data. Responder-role count (9) is stale per item 2 above (8 active today).	**[DOC-2026-09]** for municipality count; **[SCHEMA]** for the role-count correction

What this means for the manuscript

1. **The single most important correction**: any manuscript section that cites the 2026-09 report's SOS-referral live-verification test (§06) as evidence that "an SOS alert is only visible to its assigned

The manuscript should describe both models explicitly and attribute the SOS model's current, unconditional, province-wide design to `module19`'s own documented rationale (faster emergency response; no station/precinct concept exists to assign by), not as an oversight.

2. **The 2026-09 live-RLS test run itself remains legitimate, historical evidence that a real verification methodology was used on this project** — it should not be discarded, but it must be captioned as testing a **prior** mechanism, with a note that the current SOS mechanism has not had an equivalent live test performed against it (this is one of the items carried into `05-unresolved-evidence-and-testing-register.md`).
3. Every numeric claim in §04 (26 screens, 9 roles, 5 Edge Functions, 8 municipalities) needs the manuscript to cite the **current** counts from `01-deliverable-1-project-structure.md`, not the 2026-09 report's counts.
4. §07's five recommendations give the manuscript a ready-made, honest "what was recommended vs. what actually happened" narrative for a limitations/future-work chapter — items 6, 7, and 9 above are confirmed not actioned, item 8 was actioned differently than proposed, and item 10 is unverified either way. This is more defensible than presenting the recommendations as either all-resolved or ignoring them.
5. Nothing in this reconciliation required or involved access to the live Supabase project — all of it is a direct comparison between one HTML document and the current migration files/source code, both fully available on disk. The live-database-access gap (outstanding question (c), still unresolved — see `05-unresolved-evidence-and-testing-register.md`) is a separate, unresolved limitation and should not be conflated with this reconciliation.

What was explicitly not re-litigated in this document

§01 ("Background & Problem"), §02 ("Objectives"), and §03 ("Methodology") were read in full during the first audit pass, and `00-audit-scope-and-access.md` already concluded their architecture/methodology descriptions "remain substantively accurate." This document did not re-read or re-verify those three sections a second time — only §04 through §07, which the governing master prompt for this continuation specifically identified as having been seen by header only, not full content, in the first pass.

SECTION 04

ISO/IEC 25010 Questionnaire-to-Feature Traceability Matrix (Draft — Edition Unconfirmed)

Status: DRAFT. The required ISO/IEC 25010 edition/model has NOT been confirmed against ASCOT's actual manuscript requirement. Do not treat this document's characteristic list as approved.

Source document date: 2026-10-13

Source file: `docs/capstone-evidence/04-iso25010-questionnaire-traceability.md`

This is a draft working document, not an approved evaluation instrument

The governing master prompt for this audit was explicit: *"Verify the ISO/IEC 25010 edition required by ASCOT before finalizing the quality characteristics. Do not silently assume that a previously proposed model is the approved one."* This audit has **no access to any ASCOT manuscript template, rubric, or capstone-adviser guidance document** — a repository-wide search for "25010," "questionnaire," "IT Expert," "End User," and "Likert" found **zero** matching files anywhere in this repository, including the `docs/` directory itself. There is therefore **no evidence available to this audit, in this environment, of which ISO/IEC 25010 edition or quality-model structure ASCOT actually requires.** This gap cannot be closed by reading more code — it requires the capstone team to check with their adviser/ department and bring back the answer.

What is publicly known about the standard, stated here only as background for that conversation, not as a finding from this codebase: ISO/IEC 25010 has at least two published editions that structure the product-quality model differently — the original 2011 edition (8 characteristics: Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability) and a 2023 revision that restructured the model and added a 9th top-level characteristic (Safety) along with changes to some sub-characteristics. **This audit did not verify which edition, if either, ASCOT's own program requires**, and this document does not assume one is correct. The 8-characteristic list used below is used **only** because it is the one the governing master prompt itself supplied as "a working framework" — it is carried forward labeled explicitly as unverified, not silently treated as settled.

Action required from the team, not resolvable by this audit: confirm with the ASCOT capstone adviser/department which ISO/IEC 25010 edition and characteristic set the manuscript rubric actually expects, before finalizing either questionnaire. If the 2023 edition (or any other variant) is required, the "Safety" characteristic and any sub-characteristic renaming will need a corresponding row added/relabeled below — the traceability *method* in this document does not change, only the characteristic labels would.

How to read the matrix below

For each (still-unconfirmed) characteristic, this table lists **candidate** questionnaire items for two respondent groups — **IT Expert** (a technical reviewer assessing the system directly) and **End User** (a role-appropriate staff user or, where feasible and ethical, a non-identifying survivor-side reviewer) — and maps each candidate item to:

- a **specific, already-verified-or-flagged-unverified feature** from `01-deliverable-1-project-structure.md` / `02-deliverable-2-feature-matrix.md` (cited by file and section, not re-described from scratch),
- an **evaluation method** (what the respondent would actually need to do — use the UI, review a report, attempt a live test),
- the **evidence this audit already has**, versus what is still missing, and

- **limitations** — in particular, flagging every item that can currently only be evaluated as "the UI element exists and is reachable," not "the underlying behavior is correct," per the master prompt's explicit instruction not to infer successful runtime behavior from source code alone.

No item below should be read as "validated" — this is a traceability *mapping*, not a completed survey instrument, and it has not been reviewed or piloted with any respondent.

Functional Suitability

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"Each role's data access matches its documented jurisdiction (barangay/municipality/province/shelter/assignment)."	`02-*.md` §2 (RBAC/RLS), §5–§7 (case, shelter, police scoping)	Live RLS test per role, per table, with disposable accounts and real JWTs	Policy **text** read directly this pass; only the 2026-09 report's narrower 8-scenario SOS/case-referral run is actual live-test evidence, and per `03-prior-documentation-reconciliation.md` item 5 that run now tests a **superseded** SOS mechanism	Cannot be answered "yes" from source reading alone — this is exactly the gap item 3 in `02-*.md`'s unresolved-questions section describes
IT Expert	"An incident report, once submitted, produces a case record with the correct status history."	`02-*.md` §3	Submit a real test report against a live/staging project, inspect the resulting rows	Code path read and understood; **no execution evidence**	Source-code-only — explicitly flagged as unverified behavior per the governing instruction
End User (staff)	"I can find, assign, and update a case through its full lifecycle without needing to guess at an undocumented step."	`02-*.md` §5	Task-based usability walkthrough (not a test of the schema, a test of the screen)	None — this is a UX judgment that requires a real user, not a code reader	Cannot be estimated from code; must come from an actual respondent
End User (shelter admin)	"Shelter requests/referrals addressed to my shelter are clearly distinguished from other shelters' cases."	`02-*.md` §6	Live use, or a walkthrough with a seeded test account scoped to one shelter	RLS text confirms the *access* boundary; the *clarity* of the distinction is a UX judgment, not something RLS proves	Access-control correctness and UI clarity are two different claims — don't conflate them in the questionnaire wording

Performance Efficiency

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"The admin reporting/analytics pages load acceptably with a realistic data volume."	`02-*.md` §10 (Reports/Analytics)	Load test or manual timing against a seeded dataset of realistic size	None — this audit confirmed the queries are real (not static/placeholder) but never measured timing, and has no live project to measure against	Pure code reading cannot produce a performance number; this item is unanswerable without an actual environment
IT Expert	"The admin app's production bundle size is reasonable for its target users' connectivity."	`03-*.md` item 10 (2026-09 report's §07 recommendation #5, re: chunk-size warnings)	Run `vite build`, inspect the chunk-size warning output	Not re-verified this pass — flagged as a **[GAP]** in `03-*.md`	Requires actually running the build, not reading the report's claim about it

Compatibility

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"The mobile app behaves correctly across the Android OS versions/screen sizes the target population actually uses."	Not covered in Deliverable 1/2 — out of this audit's scope	Device-matrix manual testing	None in this repository	This audit has no physical-device test evidence of any kind, by design (no device access in this environment, consistent with earlier phases of this project)
IT Expert	"Push notifications integrate correctly with the OS notification center once a real device/EAS build exists."	`02-*.md` §9	Real-device test once an EAS project is configured	None — `02-*.md` §9 explicitly states push is not currently operable end-to-end (no EAS project configured)	Cannot be evaluated at all until the EAS-project prerequisite is resolved — this should be sequenced, not attempted prematurely

Usability

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
End User (survivor-side proxy or staff)	"The SOS flow is clear about what it does and does not do (e.g., it does not call police directly)."	`02-*.md` §4, "Known limitations" row	Task-based usability test with think-aloud	Code confirms no call/SMS integration exists; whether the **UI communicates this honestly to a real user** is untested	The manuscript must not claim SOS "calls police" per `02-*.md` §4 — the questionnaire item should test comprehension, not just satisfaction
End User (all staff roles)	"I always know which action buttons I'm allowed to use before I click them."	`02-*.md` §5, "Required permissions" row (buttons are not hidden by ownership tier; RLS is the real gate, discovered only on write)	Usability walkthrough across at least two roles with different case-ownership scopes	This audit found the *code reason* this could be a problem (buttons aren't conditionally hidden) but has no user-reaction evidence	A real respondent may report this as confusing or may not notice it at all — don't presume the answer from the code finding
End User (shelter admin)	"I can tell whether my last status update/occupancy change actually saved."	`02-*.md` §6, "Validation / error handling" row (shared hook never checks `{ error }`, reports success regardless)	Deliberately induce a failure (e.g., revoke network mid-save) during a usability session	Code-level gap confirmed; user-facing consequence not yet observed	This is a case where the *code finding* strongly predicts a *usability* problem, but the questionnaire should still verify it happens, not assume it

Reliability

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"A failed write (network loss, RLS denial) is surfaced to the user rather than silently appearing to succeed."	`02-*.md` §5 (`submitStatusChange` / `submitAssignment`), §6 (shared shelter hook), §9 (admin mark-read)	Deliberate failure injection, observed directly	Multiple specific **[CODE]**-confirmed gaps already identified — this characteristic has unusually strong existing evidence compared to others in this table	Still code-level findings, not yet reproduced live; listing the exact file/function for each makes this the easiest item to actually test

IT Expert	"A backup can actually be restored successfully."	`02-*.md` §10	A real, supervised backup→restore execution against a staging copy	**None** — explicitly flagged as "Not verified (insufficient evidence)" in `02-*.md` §10	Do not let a respondent "assume yes" from the code's careful design — the questionnaire should ask whether this was *demonstrated*, not whether the code *looks* careful

Security

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"Row-level security, not the UI, is the actual access-control boundary, and this holds under direct API calls."	`02-*.md` §2	Direct REST calls bypassing the UI, with real JWTs per role	Policy text read; **[DOC-2026-09]** historical test exists for one slice only, and per `03-*.md` the SOS slice of it is now stale	Same gap as item 3 in `02-*.md`'s unresolved-questions — this is the highest-value missing evidence for the whole manuscript's security chapter
IT Expert	"The AI provider key and other secrets are never exposed client-side."	`02-*.md` §8	Direct inspection of client bundle / Edge Function source (already done)	**[CODE]** — confirmed this pass: `AI_API_KEY` is read only server-side in the Edge Function, never returned to the client	This one item has strong, direct evidence already — a rare case where "yes" is well-supported, not just assumed
IT Expert	"Audit logging is complete for every security-relevant action."	`02-*.md` §10, "Known limitations" row	A full line-by-line mutation-vs-`logAudit()`-call-site audit	Partial — 31 call sites catalogued, but explicitly **not** exhaustively verified against every mutation in the codebase	Must not be scored as "complete" — the audit itself says this is unverified, not confirmed-complete

Maintainability

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"Retired roles/features are cleanly removed rather than left as dead code."	`02-*.md` §6, "Known limitations" (retired `shelter_staff` role, intentionally kept per its own migration's header comment)	Code review	**[SCHEMA]** confirmed this is a deliberate, documented decision, not oversight	A questionnaire item here should ask whether the team finds this an acceptable tradeoff, not whether dead code "exists" — it exists on purpose
IT Expert	"Error-handling patterns are consistent across similar features."	`02-*.md` §5–§9 (inconsistent try/catch and toast coverage noted in nearly every section)	Code review across the 10 features in Deliverable 2	Strong **[CODE]** evidence of inconsistency, catalogued per-feature	This is one of the best-evidenced maintainability findings in the whole audit — safe to cite directly

Portability

Respondent	Candidate item	Verified feature (source)	Evaluation method	Evidence available	Limitations
IT Expert	"A fresh deployment of this repository onto a new Supabase project would reproduce the current production environment."	`01-deliverable-1-project-structure.md`, "Storage buckets" (resolved `avatars`-bucket gap)	Attempt an actual fresh deployment	**[CODE + SCHEMA]** confirmed the `avatars` bucket is dashboard-only and would be missing on a fresh deploy; this is a real, specific, named reproducibility gap	This is a concrete, already-actionable finding — not a vague portability concern

What this document does not do

- It does not claim to be the final questionnaire — it is a traceability **scaffold** for whoever drafts the actual IT Expert and End User instruments, so that every question they write can point at a specific, already-gathered piece of evidence (or an explicit gap) instead of being invented fresh.
- It does not resolve the ISO/IEC 25010 edition question — that remains entirely with the team and their adviser.
- It does not convert any "Known limitations" entry into a pass/fail judgment — ISO/IEC 25010 evaluation is the team's and the respondents' job, not something this read-only code audit can perform on their behalf.

SECTION 05

Unresolved Evidence and Testing Register

Status: Consolidated across Deliverables 1–2 and the reconciliation pass; living document — update as items are resolved, don't delete resolved rows, mark them resolved instead
Source document date: 2026-10-13
Source file: `docs/capstone-evidence/05-unresolved-evidence-and-testing-register.md`

This is the single consolidated list the governing master prompt asked for: "which previous findings require runtime testing or additional evidence." Every row below traces back to a specific section of `01-deliverable-1-project-structure.md`, `02-deliverable-2-feature-matrix.md`, or `03-prior-documentation-reconciliation.md` — nothing here is a new claim; this document only re-sorts and prioritizes what those three already identified, so the team has one place to work from instead of ten "Additional testing required" rows scattered across files.

Priority is about **manuscript risk and system risk**, not difficulty: **Critical** items are things the manuscript cannot honestly claim are secure/working without evidence, or things that would be genuinely harmful if wrong in production. **High** items materially affect specific feature claims. **Medium/Low** items are real but narrower or cosmetic.

Critical

#	Item	Why it matters	What would resolve it	Source
C1	No live, read-only access to the Supabase project exists in this audit environment, so **no RLS policy has been confirmed to behave as written against a real request** — every policy conclusion in this entire audit is sourced from migration files, not live introspection.	This is the largest single gap behind the manuscript's Chapter 3 security claims. All 8 roles × ~15 sensitive tables have never been exhaustively, live-tested; only one narrow slice (the 2026-09 report's 8 SOS/case-referral scenarios) has real historical test evidence, and per item C2 below, part of that is now stale.	A scripted (not manual) regression suite run with disposable test accounts against the live project, covering at minimum: unassigned-record denial, cross-role denial, cross-jurisdiction denial, unauthenticated denial, and a guessed-ID probe, for every role × every sensitive table. This was also the 2026-09 report's own §07 recommendation #2, still not actioned.	`02-*.md` §2, §5–§10 (every "Additional testing required" row); `03-*.md` item 7

	run on record (2026-09 report §06) was performed — meaning the current SOS mechanism has **no** live test evidence at all, only the superseded one.	A manuscript or defense panel could be shown 8 PASS results and reasonably assume they describe the current system. They do not, for the SOS pathway specifically.	A fresh live test specifically confirming: (a) any active `law_enforcement` account can see any SOS alert province-wide (the new, intended behavior), and (b) whether this is actually desirable/acceptable to state as a finding, not just a code fact.	`02-*.md` §7; `03-*.md` item 5
C3	Backup and restore (`backup-create` / `backup-restore` Edge Functions) have **no execution evidence of any kind** — no log, no test, no record anywhere in the repository that a restore has ever been run, despite unusually careful code (checksums, mandatory safety backup, atomic compare-and-swap).	Backup/restore is the kind of infrastructure where "the code looks careful" and "it has been proven to work" are very different claims, and conflating them in a manuscript would overstate system reliability for exactly the feature meant to protect against data loss.	A real, supervised backup-create → validate → restore (begin/confirm/execute) run against a **staging**, not production, copy of the database, including one deliberately corrupted/expired backup to confirm validation actually blocks a restore.	`02-*.md` §10
C4	Supabase point-in-time recovery (PITR) is confirmed disabled on the project (per a code comment documenting a one-time live check), meaning there is **no full-database recovery mechanism at all** for survivor/case/SOS/verification data — only the custom backup module's administrative-reference-data subset is covered.	This is a real operational risk independent of the manuscript, not just a documentation gap — if it's still true today, the team should know regardless of capstone timelines.	Re-check PITR status directly on the live Supabase project dashboard (Settings → Database → Backups) and enable it if the team has the ability to, independent of this audit.	`02-*.md` §10; `03-*.md` item 6
C5	Whether the 32 SQL migration modules in this repository have actually all been applied, in order, to the live Supabase project — i.e., whether the **deployed** schema matches the **repository's** schema — remains completely unverified.	The manuscript should describe the deployed system, not the intended one. Every schema/RLS claim in this audit is a claim about the files, not necessarily about production.	A read-only schema dump or migration-history query against the live project (e.g. `supabase migration list`, or a schema-only `pg_dump`), compared against the repository's migration list.	`00-*.md`; `02-*.md` unresolved-questions item 3

High

#	Item	Why it matters	What would resolve it	Source
H1	Case status transitions (`CASE_STATUS_FLOW`) are enforced only in the admin UI's TypeScript, with **no** corresponding database trigger/CHECK constraint — unlike the comparable transition guards that do exist for `expert_support_requests`, `sos_alerts`, and `barangay_action_requests`.	A direct API call bypassing the admin UI could set a case to any status in any order, limited only by RLS ownership, not sequence.	Confirm via a direct (non-UI) API write whether an invalid sequence is actually accepted; if so, decide whether a DB-level guard is worth adding.	`02-*.md` §5
H2	Several write paths silently report success to the admin even when the underlying Supabase write fails: `submitStatusChange` / `submitAssignment` in `CaseDetailPage.tsx` (no error check at all), and `updateRequestStatus` / `updateReferralStatus` / `bumpOccupancy` in the shared shelter hook (`useShelterAdminData.ts` , shows a success toast regardless of the `{ error }` result).	A staff user could believe a case was reassigned or a shelter placement confirmed when it was not, with no indication anything went wrong.	Reproduce with a deliberate failure (e.g., revoke the relevant RLS grant temporarily on a test account, or simulate a network drop) and observe what the UI actually shows.	`02-*.md` §5, §6
H3	Shelter `current_occupancy` is a fully manual integer with no DB-level consistency mechanism and no "discharge" workflow to decrement it — it can only drift further from reality over time, and a race condition is plausible since the increment is a read-then-write with no row lock found.	Shelter capacity is operationally important (it is how a referral/request decision gets made) and currently has zero integrity guarantee.	A concurrency test (two simultaneous acceptances) and a decision on whether a "discharge" workflow or an atomic increment/trigger is worth adding.	`02-*.md` §6

		first; testing anything about push delivery before this is pointless.	Configure an EAS project, then run one real end-to-end test (register a token on a real device, trigger a high-priority notification, confirm receipt).	`02-*.md` §9
H5	`admin/src/lib/permissions.ts` (the UI-layer permission map) and the actual RLS policies have never been systematically checked for drift across all 8 roles — the file itself documents that it is "a UI-layer convenience, not the actual security boundary," but whether it and RLS agree in every case is unverified.	A drift in either direction is either a confusing-but-safe UX issue or, more seriously, a UI element that looks available but is silently rejected — neither has been checked.	A role-by-role, page-by-page comparison of what `permissions.ts` allows vs. what the corresponding RLS policy allows.	`02-*.md` §2
H6	`logAudit()` coverage across the codebase (31 known call sites across 17 files, cataloged this pass) has **not** been exhaustively checked against every mutating action in the codebase — audit completeness is "substantial" by observation, not "complete" by verification.	The audit log's own UI already discloses it is not tamper-evident; if it is also incomplete, that compounds the limitation and the manuscript should state both precisely rather than either one being assumed away.	A full, mechanical pass cross-referencing every `supabase.from(...).insert/update/delete(...)` call site in `admin/src` against the 31 known `logAudit()` call sites, to find any mutation with no corresponding audit entry.	`02-*.md` §10
H7	No jurisdiction/station/precinct restriction exists anywhere in the law-enforcement access model — any active officer can see any SOS alert province-wide. This is a documented, deliberate design choice (module19's own header), not an oversight, but it has never been evaluated as a policy decision by the team outside the code comment that justifies it.	This is exactly the kind of tradeoff a capstone panel is likely to ask about directly — "why can an officer in one town see an alert from another?" — and the answer should come from the team's own judgment, informed by this finding, not be discovered live during a defense.	A team discussion/decision, documented, on whether this tradeoff is acceptable for the final manuscript, independent of any code change.	`02-*.md` §7

Medium

#	Item	Why it matters	What would resolve it	Source
M1	`ShelterAdminDashboard.tsx` links to `/admin/shelter/services`, which does not appear as a registered route anywhere in `App.tsx`.	Possibly a broken/dead link left over from a page consolidation — or possibly resolved by a mechanism this audit didn't find.	Click the link in a running instance of the app and observe whether it 404s.	`02-*.md` §6
M2	`CaseDetailPage.submitLawEnforcementReferral` logs referral-creation failures to the console only, with no user-facing toast, unlike its sibling SOS/case-referral handlers which do show toasts.	A staff user attempting a police referral that silently fails would see nothing.	Add the same toast pattern already used elsewhere in the file, then verify.	`02-*.md` §7
M3	The admin-side `NotificationsPage.tsx` has no Realtime subscription (manual refresh only) and no rollback of its optimistic "mark read" update on failure, unlike the mobile notifications screen which has both.	A staff user could see a stale inbox or believe an item was read when the write actually failed.	Compare the two screens directly and decide whether to bring the admin page up to the same standard.	`02-*.md` §9
M4	Delivery confirmation for push notifications only goes as far as Expo accepting the push *ticket* — the Edge Function never polls Expo's separate push *receipt* API, so a "sent" status doesn't prove the device actually displayed anything.	Overstates delivery confidence if not caveated.	Decide whether receipt polling is worth adding, or document the limitation as-is.	`02-*.md` §9
M5	`SafeHer_System_Documentation_Report.html` §04's claims of "26 mobile screens" and "8 municipalities" were not independently re-counted this pass (only the role count and Edge Function count were re-verified and found stale).	Minor, but any manuscript text citing exact counts should cite current counts, not the report's.	A direct recount of `mobile/src/screens/` and the `municipalities` reference table.	`03-*.md` item 1, item 13

Low / informational

#	Item	Why it matters	What would resolve it	Source
L1	`SafeHer-System-Inventory.pdf` (repo root) remains unreadable in this environment (no PDF-rendering tool installed) — its contents have never been reflected in any pass of this audit.	Unknown content; could contain anything from a duplicate of already-known information to something materially new.	A team member with normal PDF access should open it and report back, or confirm `winget install --id oschwartz10612.Poppler` (suggested earlier this session) was applied so a future pass can read it directly.	`00-*.md`; `02-*.md` unresolved-questions item 5
L2	The ISO/IEC 25010 edition/characteristic-set ASCOT actually requires has not been confirmed — the 8-characteristic list used in `04-iso25010-questionnaire-traceability.md` is explicitly an unverified working draft, not an approved model.	Finalizing a questionnaire against the wrong edition would require redoing it.	Ask the capstone adviser/department directly; update `04-*.md`'s characteristic list accordingly (the traceability method itself would not need to change).	`04-*.md` (entire document)
L3	`shelter_staff` is a retired-but-intentionally-retained role, with its enum value, helper functions, and UI conditionals still present throughout `admin/src` by deliberate design (per its own migration's header comment).	Not a defect, but worth the manuscript describing accurately as an intentional backward-compatibility decision, not dead code the team "forgot" to remove.	No action needed beyond accurate description in the manuscript.	`02-*.md` §6
L4	Whether a `shelter_admin` can truly not read/write another shelter's rows at the RLS layer — vs. only being hidden from it by client-side filters — has not been live-tested; the client-side filter and RLS text appear consistent on paper but were not cross-checked live.	Same category as C1/C2 but narrower in scope — folded here rather than duplicated as its own Critical item.	Covered by the same regression suite described in C1, scoped to the `shelters` / `shelter_requests` / `shelter_referrals` tables specifically.	`02-*.md` §6

How to use this register going forward

- When an item is resolved, **mark it resolved in place** (e.g., prefix the row's first cell with "**RESOLVED (2026-XX-XX):**" and a one-line pointer to the evidence) rather than deleting the row — this preserves the audit trail of what was checked and when, consistent with the evidence-tagging discipline used throughout this audit.
- New items discovered in a future pass should be added with the same five-column shape and slotted into the appropriate priority tier, not appended to the bottom regardless of severity.
- This register does not replace the "Additional testing required" row in each Deliverable 2 feature table — it exists so the team has one prioritized view across all of them, not so the per-feature detail can be deleted.